Week 4 - Problem Solving

Daniel Alyoshin

CSCA48 - TUT002

January 30, 2026

Content Covered so Far

- C programming
- Memory model
- Arrays and strings
- Pointers

Review - Arrays

- Collections of contiguous boxes with the same data type.

Review - Arrays

- Collections of contiguous boxes with the same data type.
- Only first first locker (element/entry) in the array gets tagged with the array name and type (don't forget to append `[]`).

Review - Arrays

- Collections of contiguous boxes with the same data type.
- Only first first locker (element/entry) in the array gets tagged with the array name and type (don't forget to append `[]`).
- Size is fixed and holds `junk` until initialization.

Review - Arrays

- Collections of contiguous boxes with the same data type.
- Only first first locker (element/entry) in the array gets tagged with the array name and type (don't forget to append `[]`).
- Size is fixed and holds `junk` until initialization.
- Indexing: using an offset to find a specific box by counting from the first entry in the array.

Review - Arrays

- Collections of contiguous boxes with the same data type.
- Only first first locker (element/entry) in the array gets tagged with the array name and type (don't forget to append `[]`).
- Size is fixed and holds `junk` until initialization.
- Indexing: using an offset to find a specific box by counting from the first entry in the array.
- Warning! C will not prevent you from trying to access memory addresses outside of the bounds of the array.

Review - Strings

- Array of `char`

Review - Strings

- Array of `char`
- No built-in functions for dealing with strings.

Review - Strings

- Array of `char`
- No built-in functions for dealing with strings.
- Declared as an array of reasonable size which may be larger than the string itself.

Review - Strings

- Array of `char`
- No built-in functions for dealing with strings.
- Declared as an array of reasonable size which may be larger than the string itself.
- '\0' delimiter added at the end of our valid text so we can identify what is `junk` and what is actually part of the content we want.

Review - Pointers

```
int a = 5;
int *p = &a;
```

1. `a = 5`
2. `&a` = address of `a`
3. `p` = address of `a`
4. `*p = 5`
5. `&p` = address of `p`

Problem Solving Steps

Problem Solving Steps

1. Understanding the problem (this is key, do not proceed until you fully understand what you're supposed to do)

Problem Solving Steps

1. Understanding the problem (this is key, do not proceed until you fully understand what you're supposed to do)
2. Think through the solution on paper – write down the steps required to solve the problem in enough detail that you can implement the program from your written description

Problem Solving Steps

1. Understanding the problem (this is key, do not proceed until you fully understand what you're supposed to do)
2. Think through the solution on paper – write down the steps required to solve the problem in enough detail that you can implement the program from your written description
3. Design the solution (figure out how to organize the solution into functions and what each of these will do – we do not hack code until it works – we really do not start working on the code until we have a fully designed solution!

Problem Solving Steps

1. Understanding the problem (this is key, do not proceed until you fully understand what you're supposed to do)
2. Think through the solution on paper – write down the steps required to solve the problem in enough detail that you can implement the program from your written description
3. Design the solution (figure out how to organize the solution into functions and what each of these will do – we do not hack code until it works – we really do not start working on the code until we have a fully designed solution!
4. Implement the solution (which by this point will be pretty straightforward)

Problem Solving Steps

1. Understanding the problem (this is key, do not proceed until you fully understand what you're supposed to do)
2. Think through the solution on paper – write down the steps required to solve the problem in enough detail that you can implement the program from your written description
3. Design the solution (figure out how to organize the solution into functions and what each of these will do – we do not hack code until it works – we really do not start working on the code until we have a fully designed solution!
4. Implement the solution (which by this point will be pretty straightforward)
5. Thoroughly test the program – we must ensure it works properly

Problem Solving Steps - Simplified

1. Understand the problem
2. On paper write down the steps to solve it
3. Design the solution (organize it into functions)
4. Implement the solution
5. Thoroughly test the program

Your Task for Today

Some tricky hacker got into our eBook storage and messed with it. We want to write a program that will set things right.

The hacker wrote a program that went over the text of each eBook, and for every word they swapped the **second** and **fourth** letter.

We need to write a function that will accept a string of text and will **reverse the swap** that the hacker did in order to recover our original text.

Reminders

- Each student to submit their own file (picture or scan of your work) which can be the same as that submitted by members of their group.
- Ensure your file is named **exactly** as instructed on Quercus.
- You are **not** supposed to keep working on this, you will not be marked on correctness so as long as you submit work that shows you were engaged during this tutorial you will get full marks.

Watch out!

If you require multiple images to submit all of your work please place them into a **single document** and submit that.